

THE INTERVIEW GUIDE FOR TRUEWEAR

F1 Tyre Degradation & Pit Strategy Decision Support System
TrackShift Innovation Challenge 2026

Prepared for: Chinmay (hackChinmay)
Project: github.com/hackChinmay/truewear-tyre-degradation

*Every section is directly traceable to the actual codebase.
No filler. No generic career advice. Pure technical prep.*

TABLE OF CONTENTS

PART 1: THE BASICS

- 1.1 Plain-Language Explanation
- 1.2 Technical Explanation
- 1.3 Technology & Library Reference (16 Technologies)

PART 2: ARCHITECTURE & DESIGN DECISIONS

- 2.1 System Architecture Overview
- 2.2 Data Layer (database/)
- 2.3 Model Layer (models/)
- 2.4 Backend Layer (backend/)
- 2.5 Frontend Layer (frontend/)
- 2.6 Key Design Decisions & Tradeoffs (8 Core Decisions)
- 2.7 Known Limitations & How to Talk About Gaps Honestly

PART 3: IN-DEPTH Q&A

- 3.1 Tier 1 -- Walkthrough Questions (5 Questions)
- 3.2 Tier 2 -- Technical Deep-Dive (7 Questions)
- 3.3 Tier 3 -- Gotcha & Stress-Test Questions (8 Questions)

QUICK REFERENCE CARD (30 Key Constants & Metrics)

PART 1: THE BASICS

1.1 Plain-Language Explanation

TrueWear is a real-time decision support system for Formula 1 race engineers. It predicts when a car's tyres will lose grip and helps the team decide the best moment to make a pit stop. The challenge is that raw lap times are misleading -- as the car burns fuel and the track surface improves, lap times can actually get faster even while the tyres are wearing out. TrueWear strips away these hidden factors to reveal the true tyre wear, then uses a probabilistic model to forecast when the tyres will hit a critical performance cliff, and simulates hundreds of possible race scenarios to recommend the optimal pit strategy.

1.2 Technical Explanation

TrueWear is a physics-informed, online-adaptive tyre intelligence and pit strategy decision-support system for F1 motorsport. It implements a 6-term latent lap-time decomposition equation that decouples observed timing data into isolated physical components: mechanical polymer degradation, fuel mass burn-off compensation, track rubber evolution, thermal hysteresis, aerodynamic dirty-air wash, and stochastic driver variance. The isolated degradation signal feeds into an Online Adaptive Kalman State Estimator (scalar 2x2 state: degradation rate + systematic bias) that recursively updates its beliefs as each lap completes, achieving 0.084s MAE with 1.4ms inference latency. Uncertainty is quantified via heteroscedastic P10/P50/P90 intervals and cumulative Gaussian cliff probabilities using the Abramowitz & Stegun rational approximation. A 500-iteration Monte Carlo strategy engine with Box-Muller Gaussian sampling evaluates competing pit strategies under stochastic tyre degradation, continuous non-linear thermal cliff penalties, and traffic re-entry modeling. Validation follows a two-pillar framework: Pillar A (internal benchmarking against 5 candidate architectures) and Pillar B (zero-leakage chronological blind replay on unseen historical race data from the 2024 Bahrain GP via FastF1).

1.3 Technology & Library Reference

Below is every technology used in TrueWear, explained from first principles so you can describe each one even if the interviewer has never used it.

React 19 (react, react-dom)

React is a JavaScript library for building user interfaces using a component-based architecture. Components are reusable pieces of UI that manage their own state. React 19 introduces the new compiler, server components, and improved concurrent rendering. In TrueWear, React renders all 9 telemetry modules as composable components sharing state through a centralized Context provider.

TypeScript 5.8 (typescript)

TypeScript is a statically-typed superset of JavaScript. It adds compile-time type checking, interfaces, and generics to catch bugs before runtime. In TrueWear, TypeScript enforces strict contracts across 30+ domain types (TyreCompound, LapRecord, StrategyOption, LapValidationRecord, etc.), preventing runtime data shape mismatches between the math engine and the UI.

Vite 6 (vite, @vitejs/plugin-react)

Vite is a next-generation frontend build tool. Unlike Webpack, it uses native ES modules during development for instant Hot Module Replacement (HMR) -- changes reflect in <50ms without full page reloads. In production, it uses Rollup for optimized tree-shaken bundles. TrueWear runs Vite in Express middleware mode for a unified dev server on port 3000.

Tailwind CSS v4 (@tailwindcss/vite, tailwindcss)

Tailwind is a utility-first CSS framework. Instead of writing custom CSS classes, you compose visual styles directly in HTML using atomic utility classes like 'bg-gray-900', 'p-4', 'rounded-lg'. Version 4 eliminates the config file entirely -- it auto-scans your source files. TrueWear uses it for its entire dark-theme motorsport dashboard UI.

Express 4 (express)

Express is a minimal Node.js HTTP server framework. It handles routing, middleware chains, and request/response processing. TrueWear's Express server serves dual roles: it exposes REST API endpoints (/api/health, /api/strategy, /api/fastf1/*) AND hosts the Vite dev server in middleware mode for a single-port full-stack experience.

esbuild (esbuild)

esbuild is an extremely fast JavaScript/TypeScript bundler written in Go. It's 10-100x faster than Webpack for production builds. TrueWear uses esbuild to compile the backend TypeScript server (backend/server.ts) into a single CommonJS bundle (dist/server.cjs) for production deployment.

Firebase / Cloud Firestore (firebase)

Firebase is Google's Backend-as-a-Service platform. Cloud Firestore is a NoSQL document database that syncs data in real-time. TrueWear uses Firestore (project: truewear-29356) to persist strategy simulation runs -- when engineers save a Monte Carlo result, it writes to the 'strategy_runs' collection with full parameters and outcomes.

Google Gemini AI SDK (@google/genai)

The Google Gemini SDK provides access to Google's large language models for generative AI tasks. TrueWear integrates Gemini server-side to generate natural-language strategic briefings and race engineer copilot directives, translating quantitative model outputs into human-readable recommendations.

FastF1 3.8.3 (Python: fastf1)

FastF1 is a Python library that provides programmatic access to official FIA Formula 1 timing and telemetry data. It downloads, caches, and parses lap times, sector splits, car telemetry (speed, throttle, brake, gear), tyre compounds, weather, and pit stop data. TrueWear uses FastF1 to ingest real historical race data from the 2024 Bahrain GP for its Pillar B validation.

FastAPI (Python: fastapi, uvicorn)

FastAPI is a modern, high-performance Python web framework based on standard type hints. It auto-generates OpenAPI documentation and handles async I/O. TrueWear runs a FastAPI daemon on port 8000 as a microservice bridge to FastF1, since FastF1's pandas/numpy dependencies cannot run in a browser or Node.js runtime.

Framer Motion / Motion 12 (motion)

Motion (formerly Framer Motion) is an animation library for React. It provides declarative 60fps animations, layout transitions, and gesture handling. TrueWear uses it for smooth component transitions and loading state animations across all 9 dashboard modules.

Lucide React (lucide-react)

Lucide is a modern, open-source SVG icon library. Each icon is a tree-shakeable React component. TrueWear uses ~40 icons for navigation (Activity, Gauge, Settings), status indicators (AlertTriangle, CheckCircle), and telemetry labels (Thermometer, Wind, Droplets).

svg-path-properties

A utility library for computing geometric properties of SVG paths -- length, tangent angles, and point-at-distance projections. TrueWear uses it to calculate exact car positions along 2D circuit track maps based on lap progress

percentage, enabling smooth car beacon animations.

tsx (tsx)

tsx is a TypeScript execution engine for Node.js. It allows running .ts files directly without a separate compilation step, using esbuild under the hood. TrueWear's dev command ('npm run dev') uses tsx to directly execute backend/server.ts.

dotenv (dotenv)

dotenv loads environment variables from a .env file into process.env. TrueWear uses it to manage API keys (GEMINI_API_KEY), Firebase configuration, and data directory paths without hardcoding secrets.

Python Scientific Stack (pandas, numpy, scikit-learn, xgboost)

pandas provides DataFrames for structured data analysis. numpy provides fast numerical array operations. scikit-learn provides classical ML algorithms. xgboost provides gradient-boosted decision trees. TrueWear's Python backend uses these for data processing, feature engineering, and as baseline model benchmarks in Pillar A validation.

PART 2: ARCHITECTURE & DESIGN DECISIONS

2.1 System Architecture Overview

TrueWear follows a layered, modular architecture with strict separation of concerns:

- * Data Layer (database/) -- Domain types, mock datasets, circuit vectors, Firestore rules, real validation dossiers.
- * Model Layer (models/) -- Pure mathematical engines: degradation model, simulation engine, strategy simulator, validation engine. Zero side effects, fully deterministic for given inputs.
- * Backend Layer (backend/) -- Node.js Express server + Python FastAPI microservice. REST APIs, Vite middleware integration, FastF1 telemetry proxy.
- * Frontend Layer (frontend/) -- React 19 SPA with 9 telemetry modules, centralized RaceContext state management, Provider-pattern data abstraction, SVG-based visualizations.

The key architectural principle is that ALL mathematical computations happen in the Model Layer as pure TypeScript functions. The frontend never contains business logic -- it only renders model outputs. The backend never contains model logic -- it only serves data. This makes every engine independently testable and prevents state leakage.

2.2 Data Layer (database/)

types.ts -- The Domain Contract

This 270-line file defines 14+ TypeScript interfaces that serve as the single source of truth for every data shape across the entire system. Key types include:

- * TyreCompound: 'SOFT' | 'MEDIUM' | 'HARD' | 'INTERMEDIATE' | 'WET'
- * LapRecord: Full lap telemetry with decoupled latent factors (observedDelta, trueWearDelta, fuelOffset, trackGripOffset, trafficOffset)
- * WheelTelemetry: 4-corner pyrometer data (surfaceTemp, carcassTemp, pressure, wearPercent, blisterProbability, grainingStatus)
- * StrategyOption: Candidate pit plans with Monte Carlo confidence percentages and re-entry traffic analysis
- * LapValidationRecord: Per-lap blind prediction vs actual comparison with Kalman gain and P10/P90 coverage
- * HistoricalValidationMetrics: MAE, RMSE, R-squared, 95th percentile, prediction interval coverage, adaptive improvement %
- * RaceDataProvider: Interface abstraction enabling zero-code swaps between mock and live data sources

mockRaceData.ts -- Calibrated Race Datasets

Contains pre-calibrated F1 datasets for 4 Grand Prix circuits (Monza, Silverstone, Spa, Bahrain). Each circuit includes physical parameters (track length, lap count, corner count, base lap time, cliff threshold), 10-driver leaderboard grids with realistic intervals, compound-specific confounding factor quantifications (fuel burn, track evolution, traffic wake, thermal stress), 3 strategy options (Plan A/B/C), and a synthetic 53-lap telemetry array generated using the exact physics decomposition equation.

circuitTrackData.ts -- 3,913 Lines of Vector Coordinates

Contains FIA-sourced track layouts for Monza, Silverstone, and Spa-Francorchamps as 200+ waypoint arrays. Each waypoint includes x/y coordinates, sector assignment, speed (up to 350 km/h), gear, throttle, brake, and steering angle. DRS zones, pit lane paths, and corner annotations (names, turn numbers, apex notes) are included. The `getCircuitTrackTelemetry()` function linearly interpolates between waypoints based on lap progress (0.0 to 1.0) to compute smooth car positions for the interactive circuit map.

realWorldValidationBahrain.ts -- 7,898 Lines of Real FIA Data

Contains the complete Pillar B validation dossier from the 2024 Bahrain Grand Prix, sourced via FastF1. Includes 57-lap blind replay results for 4 drivers (Sainz, Verstappen, Leclerc, Norris), per-lap predicted vs actual comparisons, excluded laps table (standing start, yellow flags, pit transitions), top-10 failure case taxonomy with root-cause diagnoses, and multi-driver cross-validation summary.

2.3 Model Layer (models/)

degradationModel.ts -- The Core Physics Engine

Implements the 6-term latent lap-time decomposition equation that isolates true mechanical tyre wear from confounding factors:

$$\text{delta_t}(n) = \text{delta_base} + k_wear * n^{\alpha} + \text{beta_thermal} * \max(0, T_track - 38) - \text{gamma_fuel} * \text{delta_M} - \text{delta_track} * \ln(n) + \text{epsilon_traffic}$$

Key algorithms and constants:

- * Base degradation rates: Soft 0.114 s/lap, Medium 0.078 s/lap, Hard 0.042 s/lap
- * Fuel compensation: -0.0581 s/lap (from ~1.72 kg/lap mass loss)
- * Track evolution: -0.038 s/ln(lap) (rubber deposition grip gain)
- * Thermal multiplier: +3.9%/deg C above 38 deg C reference asphalt temperature
- * Heteroscedastic P10/P50/P90 uncertainty: sigma expands with tyre age (+4.5%/lap beyond lap 8) and thermal stress (+2%/deg C)
- * Cumulative Gaussian cliff probability using Abramowitz & Stegun rational approximation (max error < 7.5e-8)
- * Cliff parameters: Soft mean=15.5 laps sigma=1.2, Medium mean=24.0 sigma=1.4, Hard mean=35.0 sigma=1.8
- * Pillar A internal benchmarks: evaluates 5 candidate architectures from naive persistence (MAE 0.382s) to TrueWear Hybrid (MAE 0.084s)

historicalValidationEngine.ts -- The Kalman Filter & Pillar B

Implements the Online Adaptive Kalman State Estimator and zero-leakage blind replay engine.

Kalman Filter State:

- * State vector: [degradation_rate, systematic_bias]
- * Initial state: d0 = 0.078 s/lap, b0 = 0.0 s
- * Initial covariance P0 = 0.04, Process noise Q = 0.0015, Measurement noise R = 0.018

Update cycle per lap:

1. TIME UPDATE (Prediction): Prior = current estimate; Covariance grows by Q
2. MEASUREMENT GATING: Exclude non-clean laps (pit transitions, yellow flags, dirty air > 0.5s). Kalman gain = 0 for excluded laps.
3. INNOVATION RESIDUAL: $y = \text{actual_lap_time} - \text{predicted_lap_time}$
4. KALMAN GAIN: $K = P_prior / (P_prior + R)$
5. STATE UPDATE: $\text{deg_rate} += K * y / \max(1, \text{tyreAge})$; $\text{bias} += K * 0.2 * y$
6. COVARIANCE UPDATE: $P = (1 - K) * P_prior$

Also implements Leave-One-Race-Out (LORO) cross-validation across 4 circuits and top-10 failure case taxonomy.

strategySimulator.ts -- Monte Carlo Strategy Engine

Runs 500-iteration stochastic Monte Carlo simulations comparing Plan A, B, and C pit strategies.

- * Box-Muller transform: generates Gaussian random shocks from uniform samples. $z0 = \sqrt{-2 * \ln(u1)}$ *

`cos(2*pi*u2)`

- * Continuous non-linear thermal penalty: when `age > cliffAge`, `penalty = (overCliff)^1.38 * 0.24`
- * Fuel burn correction: `-0.055 s/lap * (lap - currentLap)`
- * Traffic re-entry modeling: evaluates clean-air probability based on gap behind, pit loss, and race position
- * Win probability: % of iterations where each plan has the lowest total race time
- * 5-point explainable pit-wall directives: race time, degradation cliff, pit transit delta, clean air re-entry, compound transition

simulationEngine.ts -- Full Race Simulation

Provides real-time race state simulation across all 9 dashboard modules.

- * Lap time synthesis: `basePace + fuel(-0.058*lap) + grip(-0.025*ln(lap+1)) + wear(degRate*tyreAge) + driver oscillation(sin/cos)`
- * Sector split proportioning: S1=32.2%, S2=32.4%, S3=35.4% with sinusoidal micro-perturbations
- * 4-corner wheel thermal model: captures clockwise circuit lateral asymmetry (FR most stressed). Gay-Lussac pressure expansion.
- * 20-driver leaderboard with dynamic pit sequences, interval gap computation, and position changes
- * Track weather micro-climate: solar heating curve peaking at lap 31, turbulent wind oscillations
- * AI strategy signal generation: contextual BOX NOW / STAY OUT / ATTACK directives with confidence %

2.4 Backend Layer (backend/)

server.ts -- Express Full-Stack Server

Dual-role Node.js server handling both API and frontend serving:

- * GET `/api/health` -- System heartbeat with capability list
- * GET `/api/strategy` -- Pit strategy recommendations based on current lap, pit lap, circuit, and model type. Decision rules: PIT_NOW (within +/-1 of pit lap), PIT_IN_N_LAPS (within 5 laps), STAY_OUT (otherwise)
- * GET `/api/fastf1/sessions` -- Supported GP event list by year
- * GET `/api/fastf1/weather` -- Pre-calibrated environmental profiles per circuit
- * In dev: Vite middleware mode for HMR. In prod: static file serving from `dist/` with SPA fallback.

fastf1_service.py -- Python FastAPI Daemon

Microservice bridging the Python/JS boundary for FastF1 telemetry access:

- * Dual caching: disk pickle cache (`fastf1.Cache`) + in-memory session cache (dict)
- * 6+ endpoint groups: status, sessions, weather, telemetry traces (subsampling to ~60 points), driver standings, strategy recommendations
- * Graceful fallback: if FastF1 is offline, synthesizes high-fidelity 60-point telemetry profiles using harmonic sinusoids
- * Telemetry subsampling: `step = max(1, len(car_data) // 60)` for consistent 60-point arrays regardless of raw data density

2.5 Frontend Layer (frontend/)

RaceContext.tsx -- The Central Nervous System

Single React Context provider that eliminates all prop drilling. Manages 50+ state variables and actions: routing, circuit/driver/session selection, lap stepping & playback (40ms tick at 25fps), car telemetry interpolation, physics correction toggles, strategy selection, wheel telemetry, leaderboard, race events, and data provider switching.

Simulation loop runs at `BASE_LAP_MS = 1600ms` per lap at 1x speed. Progress increment per tick: $\text{delta} = \text{TICK_INTERVAL} / (\text{BASE_LAP_MS} / \text{simulationSpeed})$. When progress ≥ 1.0 , it triggers a full state synchronization: recalculates leaderboard, wheel temps, weather, strategy signal, and events.

9 Dashboard Modules (Pages)

- Module 01 CommandCenter:** Race engineer cockpit. 4 KPI cards, stint timeline, degradation chart, thermal telemetry, AI copilot directive, paddock timing matrix. Clicking a competitor focuses telemetry on that driver.
- Module 02 TyreIntelligence:** Physics decomposition. Isolates true wear from fuel/track evolution/traffic. Client-side CSV export engine. 5-lap forward predictor with heteroscedastic uncertainty and cliff probability. Interactive decoupling toggles.
- Module 03 StrategySimulator:** Monte Carlo sandbox. Adjustable sliders for lap, compound, pit lap, number of stops. Runs 500 iterations, displays Plan A/B/C with win probabilities and lap-by-lap delta curves. Saves results to Cloud Firestore.
- Module 04 LiveRaceMonitor:** Real-time replay. Interactive SVG circuit map with multi-car positioning, speed heatmap, DRS zones, sector coloring, fullscreen zoom. Docked telemetry HUD (speed, gear, throttle/brake bars, DRS, steering). Race event audit feed.
- Module 05 StintAnalysis:** Multi-compound kinetics. Detailed stint cards with stability index, terminal state, pace trends. SVG pace kinetics chart across all stints. Corner thermal energy grid (FL/FR/RL/RR heat dissipation in kJ).
- Module 06 TrackConditions:** Micro-climate analysis. Air/track temp, humidity, wind vector, grip index. Track evolution logarithmic curve. Thermal gradient vs degradation multiplier chart. 5-factor decoupled share breakdown.
- Module 07 Reports:** Engineering dossiers. Auto-generates full reports with 4 sections (Tyre, Strategy, Conditions, Model). Print/PDF export via `window.print()` with custom @media print CSS. CSV data export. Modal dossier viewer.
- Module 08 ModelDiagnostics:** ML transparency. Pillar A benchmark comparison (5 architectures). Interactive hyperparameter tuner (`wearAlpha`, `baseWearRate`, `fuelBeta`, `thermalBeta`, `trackEvoDelta`). SHAP feature attribution with engineering disclaimer. Live equation display.
- Module 09 RealWorldValidation:** Pillar B scientific validation. 6 interactive SVG visualizations (trajectory, residuals, degradation trace, cliff probability curve, P10-P90 ribbon, Model A vs B comparison). Top-10 failure case table. Multi-race LORO cross-validation.

2.6 Key Design Decisions & Tradeoffs

WHY physics-informed decomposition instead of end-to-end deep learning?

F1 race data is extremely small-sample (50-70 laps per race, varying compounds, changing weather). A deep learning model would overfit to specific sessions and fail to generalize across circuits. By using physics equations as structural priors, the model captures domain knowledge (fuel burns at a known rate, rubber deposits logarithmically) and only uses the Kalman filter to adapt the residual. Result: MAE 0.084s with sub-2ms latency vs typical DL models that need thousands of training laps.

Alternative considered: LSTM/Transformer on raw telemetry. Rejected because of insufficient training data, lack of explainability on the pit wall, and unacceptable inference latency for real-time strategy.

WHY a scalar 2x2 Kalman filter instead of an Extended/Unscented Kalman or particle filter?

The state space is inherently low-dimensional -- we're estimating two quantities: degradation rate and systematic bias. A full EKF or UKF introduces unnecessary complexity (Jacobian computation, sigma point generation) for a problem with a

linear transition model (random walk). The scalar Kalman achieves optimal Bayesian updating with $O(1)$ computation per lap and zero matrix operations. If the state space grew (e.g., estimating compound-specific rates simultaneously), an EKF would be warranted.

Alternative considered: Particle filter for multi-modal cliff onset. Rejected because cliff probability is already modeled separately via the Gaussian CDF, making particle filtering redundant.

WHY Abramowitz & Stegun polynomial instead of calling Math.erf or a library?

TypeScript/JavaScript has no built-in normal CDF function. Importing a statistics library (e.g., jStat) adds bundle weight for a single function. The Abramowitz & Stegun 5th-order rational approximation (Formula 26.2.17) achieves max absolute error $< 7.5e-8$ in a self-contained 15-line implementation with zero dependencies. It's used in the degradation model's cliff probability and the strategy simulator's confidence intervals.

Alternative considered: Numerical integration (Simpson's rule). Rejected because it requires iterative computation vs the closed-form polynomial.

WHY split the backend between Node.js Express and Python FastAPI?

FastF1 depends on pandas, numpy, and scipy -- Python-only packages that cannot run in Node.js or a browser. Rather than rewriting FastF1 in TypeScript (impractical), we use a microservice architecture: Python FastAPI daemon on port 8000 handles FIA data ingestion and caching, while Node.js Express on port 3000 serves the frontend and lightweight API endpoints. The frontend uses a Provider Design Pattern (RaceDataProvider interface) to abstract this boundary -- switching from mock data to live FastF1 requires zero code changes.

Alternative considered: WebAssembly compilation of Python scientific stack. Rejected due to maturity/size constraints.

WHY React Context instead of Redux/Zustand for state management?

TrueWear has a single global state container (RaceContext) that manages ~50 state variables. React Context with the useContext hook provides sufficient reactivity without the boilerplate of Redux (actions, reducers, selectors, middleware). The application has one page active at a time, and re-render scope is controlled by memoization. For a team of 1-2 developers, Context's simplicity outweighs Redux's tooling advantages.

Tradeoff: If TrueWear scaled to 100+ state variables with deeply nested selectors, Redux Toolkit or Zustand would reduce unnecessary re-renders via atomic selector patterns.

WHY Box-Muller transform instead of using a library for Gaussian sampling?

The Monte Carlo engine needs to generate standard normal random variates in the browser without native C-bindings. The Box-Muller transform converts two uniform random numbers (Math.random()) into Gaussian samples in 3 lines of code. It's mathematically exact, requires no lookup tables, and has zero dependencies. A library like d3-random would add 30KB+ for a single function.

Alternative considered: Ziggurat algorithm (faster per sample). Rejected because $500 \text{ iterations} \times 3 \text{ strategies} = 1,500 \text{ samples}$ -- the performance difference is sub-microsecond at this scale.

WHY continuous non-linear thermal penalty ($\text{overCliff}^{1.38} * 0.24$) instead of a hard cutoff?

Real tyre cliff behavior is progressive, not binary. A hard cutoff at the cliff lap would create discontinuous strategy landscapes where a 1-lap difference in pit timing produces wildly different outcomes. The continuous power-law penalty (exponent 1.38) models the physical reality that degradation accelerates smoothly past the cliff onset, allowing the Monte Carlo optimizer to find nuanced timing gradients.

Alternative considered: Step function or sigmoid. Rejected because both create sharp transitions that destabilize the optimizer.

WHY SVG for charts instead of a charting library (Chart.js, D3, Recharts)?

Custom SVG gives pixel-perfect control over motorsport-specific visualizations: cliff boundary lines, P10-P90 uncertainty polygons, compound color-coded stint bars, interactive hover tooltips with decoupled factors. A charting library would require extensive customization to achieve this level of domain-specific rendering. The SVG code is also more transparent for interview conversations -- every coordinate calculation is visible in the source.

Tradeoff: Higher development effort per chart. If TrueWear needed 20+ generic charts, a library would save time.

2.7 Known Limitations & How to Talk About Gaps Honestly

Interviewers respect candidates who acknowledge limitations proactively. Here are the real ones:

P10-P90 coverage is under-dispersed (37.3% on Bahrain vs target 80%)

Say: 'Our uncertainty intervals are currently calibrated for synthetic data and are too narrow for real-world variance. The fix is to widen `sigma_base` from 0.012 to ~0.035 and retune the age expansion coefficient on a larger multi-race dataset. This is an active calibration task, not an architectural flaw.'

Silverstone and Spa LORO folds show high absolute MAE (4.9s, 18.2s)

Say: 'These circuits use synthetic baseline data, not real FastF1 telemetry. The Kalman filter adapts to the synthetic data's structure, which doesn't match real-world variance. Bahrain (real data) shows 0.574s MAE. The fix is to run the FastF1 validation script on Silverstone and Spa 2024 races to generate real dossiers.'

No real-time live data integration yet -- demo mode uses mock provider

Say: 'The architecture is already designed for live data -- the `RaceDataProvider` interface and `LiveOpenF1Provider` class exist. What's missing is the `WebSocket` integration for sub-second telemetry streaming and the authentication layer for FIA's live timing feed. The mock provider lets us develop and validate the entire pipeline without a live connection.'

Single-compound Kalman state -- doesn't adapt per-compound independently

Say: 'The current Kalman state estimates a single degradation rate. When the driver pits and switches compounds, the state resets to the new compound's base rate. A more sophisticated approach would maintain parallel filter banks -- one per compound -- and switch between them at pit stops. This is straightforward to implement but wasn't needed for the current validation targets.'

No wind/rain/safety car stochastic modeling in Monte Carlo

Say: 'The Monte Carlo engine perturbs tyre degradation rates but treats weather and safety cars as deterministic. A production system would need Poisson-process safety car injection and weather transition matrices from historical data. This is a scope extension, not a missing capability.'

PART 3: IN-DEPTH Q&A

3.1 Tier 1 -- Walkthrough Questions

These are 'tell me about your project' questions. The interviewer wants to see if you understand your own system end to end.

Q: Walk me through what happens when a race engineer opens TrueWear and advances the simulation by one lap.

A: When the simulation advances from lap N to lap N+1, the RaceContext's 40ms timer tick increments the lap progress counter. Once progress crosses 1.0, it triggers a full state sync cycle. Five things happen in sequence: First, calculateDriverLeaderboard() recomputes all 20 drivers' positions, intervals, and pit statuses. Second, calculateWheelTelemetry() updates all four wheel temperatures, pressures, and wear percentages -- the front-right is always hottest on clockwise circuits due to lateral load asymmetry. Third, calculateWeather() updates the track and air temperatures along a solar heating curve that peaks at lap 31. Fourth, calculateStrategySignal() re-evaluates whether to recommend BOX NOW, STAY OUT, or ATTACK based on the current tyre age relative to the cliff threshold and the gap to the car behind. Fifth, getEventsUpToLap() filters the race event log. All of this completes in under 2ms, and every dashboard module re-renders with the new data through React's Context subscription.

[Interviewer is testing: Can you explain a real system end-to-end? Do you understand the data flow?]

Q: Explain the core mathematical model in TrueWear. What equation drives everything?

A: The foundation is a 6-term latent lap-time decomposition equation: $\Delta_t(n) = \Delta_{base} + k_{wear} * n^{\alpha} + \beta_{thermal} * \max(0, T_{track} - 38) - \gamma_{fuel} * \Delta_M - \Delta_{track} * \ln(n) + \epsilon_{traffic}$. Each term isolates a physical phenomenon: k_{wear} captures mechanical polymer abrasion, γ_{fuel} compensates for 1.72 kg/lap of fuel burn that artificially speeds up lap times, Δ_{track} accounts for rubber depositing on the racing line, $\beta_{thermal}$ models how hot asphalt accelerates compound degradation by 3.9% per degree above 38 degrees C, and $\epsilon_{traffic}$ adds the dirty-air penalty from following another car within 1.5 seconds. The key insight is that raw lap times are CONFOUNDED by these factors -- without decoupling them, you'd predict negative degradation because the car gets lighter faster than the tyres wear out.

[Interviewer is testing: Does the candidate understand the math they implemented, or did they copy-paste it?]

Q: How does TrueWear validate that its predictions are actually accurate?

A: We use a two-pillar validation framework. Pillar A is internal benchmarking -- we compare TrueWear's hybrid model against 4 alternative architectures on calibration data: naive persistence (MAE 0.382s), linear regression (0.224s), black-box polynomial ML (0.168s), and physics-only without Kalman (0.126s). TrueWear achieves 0.084s MAE, beating all baselines while maintaining 1.4ms inference latency. Pillar B is the real test -- we run a zero-leakage blind replay on the 2024 Bahrain Grand Prix using actual FIA timing data from FastF1. The model processes each lap chronologically, predicts the next lap, and only then sees the actual result. On an unseen circuit with 4 different drivers, we achieved 0.574s MAE for Sainz, 0.420s for Verstappen, and demonstrated 49-65% accuracy improvement over the frozen offline model. We also show that 100% of our pit stop predictions fell within plus or minus 1 lap of the actual team pit calls.

[Interviewer is testing: Does the candidate understand validation methodology? Can they distinguish train vs test?]

Q: What is the Kalman filter and why did you choose it for this project?

A: A Kalman filter is an optimal Bayesian state estimator for linear-Gaussian systems. It maintains a belief about the true state of a system -- in our case, the degradation rate and systematic bias -- and updates that belief each time it receives a new measurement. It works in two steps: the PREDICT step (where the state propagates forward under the process model and uncertainty grows by the process noise Q), and the UPDATE step (where a new measurement reduces uncertainty through the Kalman gain $K = P/(P+R)$, which optimally balances how much to trust the model vs the measurement). I chose it because our state space is low-dimensional (2 scalars), the transition model is a simple

random walk (the degradation rate drifts slowly), and we need online adaptation -- the model must improve its predictions as each lap completes without retraining. The Kalman filter achieves this optimally in $O(1)$ computation per lap. An important design choice is measurement gating: we skip the update on yellow-flag laps, pit transitions, and heavy-traffic laps to prevent corrupted timing data from distorting the filter state.

[Interviewer is testing: Can the candidate explain a Kalman filter from first principles? Do they understand WHY it was chosen?]

Q: Walk me through how the Monte Carlo strategy simulator works.

A: The simulator evaluates three candidate pit strategies -- Plan A (optimal 1-stop), Plan B (extended overcut), and Plan C (aggressive 2-stop attack) -- under stochastic uncertainty. For each of 500 iterations, I sample a Gaussian noise shock using the Box-Muller transform: $z = \sqrt{-2 \ln(u_1)} * \cos(2\pi u_2)$. This shock perturbs the base tyre degradation rate for each compound. Then I simulate the remaining race lap-by-lap for each plan: each lap's pace = base_pace + compound_delta + effective_wear + fuel_correction. If the tyre goes past the cliff age, a continuous power-law penalty kicks in: $(\text{overCliff})^{1.38} * 0.24$ -- this models the physical reality that cliff degradation is progressive, not a hard cutoff. At pit stops, I add the 21.4-second stationary loss and evaluate the traffic re-entry gap. After all iterations, Plan A might win 72% of the time, Plan B 18%, Plan C 10% -- giving the race engineer a probabilistic basis for the pit call, not just a single-point recommendation.

[Interviewer is testing: Understanding of Monte Carlo methods, stochastic simulation, and decision under uncertainty.]

3.2 Tier 2 -- Technical Deep-Dive

These questions probe specific code, algorithms, and implementation details. The interviewer has read your code or will ask you to explain specific functions.

Q: The Abramowitz & Stegun approximation -- what is it, why not just use Math library functions, and what's the error bound?

A: The Abramowitz & Stegun formula 26.2.17 is a 5th-order rational polynomial approximation to the standard normal cumulative distribution function $\Phi(x)$. JavaScript and TypeScript have no built-in normal CDF -- Math.erf doesn't exist. I could import jStat or mathjs, but that adds 30-100KB to the bundle for a single function. The implementation uses: $t = 1/(1 + 0.2316419|x|)$, then $\Phi = 1 - Z(x) * (b_1 t + b_2 t^2 + b_3 t^3 + b_4 t^4 + b_5 t^5)$ where $Z(x) = (1/\sqrt{2\pi}) * \exp(-x^2/2)$. The 5 polynomial coefficients are: $b_1=0.31938153$, $b_2=-0.356563782$, $b_3=1.781477937$, $b_4=-1.821255978$, $b_5=1.330274429$. For $x < 0$, we use the symmetry property: $\Phi(x) = Z(|x|) * P(t)$. Maximum absolute error is less than $7.5 * 10^{-8}$, which is far beyond the precision needed for cliff probabilities that are displayed as integer percentages. Execution is constant-time: one exponential, one division, five multiplications.

[Interviewer is testing: Deep algorithmic knowledge. Can the candidate implement numerical methods from scratch?]

Q: Explain the heteroscedastic uncertainty model. Why not just use constant error bars?

A: Heteroscedastic means 'different variance' -- the prediction uncertainty is NOT constant across the tyre's life. Fresh tyres behave predictably (low variance), but as they age past lap 8, the carcass structure starts responding non-linearly to thermal cycling, and small differences in driving style, track temperature, or rubber pickup compound into larger timing differences. My model captures this: $\sigma(n) = \sigma_{\text{base}} * [1 + 0.045 * \max(0, \text{age} - 8)] * [1 + \max(0, T_{\text{track}} - 38) * 0.02]$. σ_{base} is 0.012 s/lap. At tyre age 20 on a 42-degree track, σ expands to $0.012 * (1 + 0.045 * 12) * (1 + 4 * 0.02) = 0.012 * 1.54 * 1.08 = 0.020$ s/lap. P10 and P90 use the 1.282 z-score (80% central coverage): $P10 = P50 - 1.282 * \sigma$, $P90 = P50 + 1.282 * \sigma$. Constant error bars would be misleading -- they'd be too wide on fresh tyres and too narrow on worn ones.

[Interviewer is testing: Understanding of prediction uncertainty, calibration, and statistical rigor.]

Q: What is zero-leakage validation and how did you implement it?

A: Zero-leakage means the model NEVER sees future data during evaluation. In many ML projects, people accidentally leak future information through feature engineering, normalization, or batch processing. Our implementation is strict: For each lap N in the historical race, the model receives ONLY laps 1 through N. It generates a prediction for lap N+1 and records it. THEN -- and only then -- it reveals actual lap N+1, computes the error, and allows the Kalman filter to update.

Pit in-laps, out-laps, standing starts, and yellow flag laps are identified and placed in an excluded laps table -- they don't corrupt the error statistics but they also don't update the Kalman state. The implementation runs two model tracks in parallel: Model A (frozen offline physics, never adapts) and Model B (online Kalman, adapts each lap). Comparing them proves that the online adaptation actually helps -- on Bahrain, Model B improved MAE by 60.3% over Model A.

[Interviewer is testing: Understanding of data leakage, train/test contamination, and rigorous evaluation.]

Q: How does the Provider Design Pattern work in TrueWear's frontend?

A: The RaceDataProvider is a TypeScript interface defining methods like `getSessions()`, `getDrivers()`, `getLaps()`, `getStints()`, `getWeather()`, `getTelemetry()`, and `getProviderStatus()`. Three concrete implementations exist: `DemoRaceDataProvider` (returns mock data from `mockRaceData.ts` -- zero network calls), `FastF1HttpRaceDataProvider` (calls the Python FastAPI daemon at `localhost:8000`), and `LiveOpenF1Provider` (connects to the FIA's live timing feed). The frontend code NEVER imports a specific provider directly. Instead, `RaceContext` holds a 'dataProvider' state variable that's set by `switchDataProvider()`. All components call `dataProvider.getLaps()` etc. On startup, the app probes whether the FastF1 daemon is running -- if yes, it switches to real data; if not, it falls back to `DemoProvider` with zero user intervention. This pattern means we can develop the entire UI without a Python backend, demo at competitions without network access, and connect to live data in production -- all without changing a single component.

[Interviewer is testing: Design pattern knowledge, abstraction, and practical software architecture.]

Q: Explain the measurement gating logic in the Kalman filter. What gets excluded and why?

A: Three types of laps are gated out: First, pit transitions -- when `tyreAge` equals 1 or the lap is flagged as `inPitWindow`. Pit in-laps include deceleration into the pit lane speed limit (80 km/h), and out-laps have cold tyres and reduced grip. These timing anomalies are 15-25 seconds off normal pace and would violently distort the filter's degradation rate estimate. Second, yellow flag or safety car laps -- when the flag is not GREEN. The entire field slows to a mandated delta time, making the lap time meaningless for degradation estimation. Third, heavy traffic laps -- when `trafficOffset` exceeds 0.5 seconds. Being stuck behind another car in dirty air causes aero-wash-induced lap time loss that is NOT related to tyre degradation. When a lap is excluded, the Kalman gain is set to zero: no state update, no covariance reduction. The prior state carries forward unchanged. This preserves filter integrity through anomalous race events.

[Interviewer is testing: Domain-specific engineering judgment, not just textbook Kalman filter implementation.]

Q: Walk me through the 4-corner wheel thermal model. Why is the front-right always hottest?

A: On clockwise circuits like Monza, Silverstone, and Spa, the majority of corners load the front-right tyre. At Monza specifically, Curva Grande, Roggia, Lesmo 1 & 2, Ascari, and Parabolica are all right-handers that drive continuous lateral energy into the FR compound. The model captures this asymmetry explicitly: FR baseline surface temp is 98 degrees C with +0.25 degrees/lap, reaching 108 degrees C by lap 40 -- triggering THERMAL PEAK status. FL runs at 99 degrees C baseline but with slower +0.18 degrees/lap aging. Rears run 97-98 degrees C with even lower expansion rates. Carcass core temperature is modeled as $T_{\text{surface}} - 4 \text{ degrees C}$ (bulk thermal lag). Pressure follows Gay-Lussac's law approximation: $P = P_{\text{cold}} + (T_{\text{surface}} - 95) * \beta_{\text{pressure}}$, where β is 0.06 PSI/deg C on FR (higher load = higher thermal sensitivity) and 0.04 PSI/deg C on rears. Blister probability on FR reaches 1.4% per lap of tyre age, while rears stay under 0.6%.

[Interviewer is testing: Physical understanding of tyre dynamics, not just code description.]

Q: How does RaceContext manage the 25fps simulation loop without causing React performance issues?

A: The simulation loop uses a single `setInterval` at 40ms (25fps) -- this is intentionally chosen to be above the perceptual smoothness threshold (24fps) but below the 60fps cap that would waste CPU on a data dashboard. The key performance decision is that the 40ms tick ONLY updates the `lapProgress` counter (a single floating-point number) and car telemetry coordinates. This causes minimal re-rendering -- only the circuit map's car beacon moves. The EXPENSIVE state syncs (leaderboard, wheel telemetry, weather, strategy, events) only fire when progress crosses 1.0 -- roughly every 1.6 seconds at 1x speed. This batched approach means React reconciles the full state tree 30-35 times per race, not 1,000+ times. Additionally, the car position is computed via `getCircuitTrackTelemetry(circuitId, progress % 1.0)` which is a pure function -- no side effects, no state mutations, just coordinate interpolation.

[Interviewer is testing: React performance optimization knowledge, understanding of re-render costs.]

3.3 Tier 3 -- Gotcha & Stress-Test Questions

These are designed to catch you off guard, probe your intellectual honesty, and test how you handle criticism or edge cases.

Q: Your Bahrain validation shows R-squared of 0.591. That's barely above 0.5 -- isn't that a weak model?

A: You're right that 0.591 sounds low if you're comparing to a typical regression R-squared. But context matters. First, this is an UNSEEN circuit -- the model was never trained or calibrated on Bahrain. It's a cold-start generalization test, not a fitting exercise. Second, F1 lap time variance includes driver-specific factors like lift-and-coast fuel saving, energy recovery harvesting, and team radio instructions that are fundamentally unpredictable from tyre physics alone. The R-squared captures the fraction of variance attributable to tyre degradation physics. Third, the actionable metric is MAE (0.574s for an unseen circuit), not R-squared. For pit strategy decisions, predicting within half a second is operationally useful. And our pit stop timing predictions were 100% within plus or minus 1 lap of actual team pit calls -- which is the real success criterion. On calibration data (Monza), R-squared is 0.979. The gap between 0.979 and 0.591 quantifies generalization loss from domain shift, not model failure.

[Interviewer is testing: How the candidate handles criticism. Do they get defensive or explain the context?]

Q: Isn't your 'real-world validation' just validation against one race? That's a sample size of 1.

A: Strictly speaking, yes -- one race, one circuit, but four independent drivers. That's $4 \times \sim 51$ valid laps = ~ 204 independent prediction-observation pairs. But I completely agree this isn't enough for production confidence. The Bahrain validation is a proof-of-concept showing the pipeline works: FastF1 ingests real data, the blind replay protocol prevents leakage, and the Kalman filter demonstrably improves over the frozen model. The next step is to run the validation script on 5-10 more 2024 races (the FastF1 cache already supports this) and build a multi-race confidence interval. I chose Bahrain as the first test because it's the season opener -- no prior 2024 data exists for any team, making it the hardest possible generalization test.

[Interviewer is testing: Intellectual honesty about sample size. Does the candidate know the limits of their validation?]

Q: If fuel burn is -0.058 s/lap and you know the race distance, why do you need a model at all? Can't you just subtract the known offsets?

A: Great question -- and that's exactly what the 'Physics-Only' baseline in Pillar A does. It decouples known confounders and predicts using the analytical equation alone, achieving 0.126s MAE. But it fails in two scenarios: First, the base degradation rate varies session-to-session due to asphalt resurfacing, ambient conditions, and tyre batch manufacturing variance. The fixed coefficient 0.078 s/lap is a calibrated average, not a universal constant. Second, driver behavior changes mid-race -- a driver might start pushing harder after a safety car restart, or lift-and-coast to save fuel in the closing laps. The Kalman filter adapts the degradation rate online using observed residuals, closing the gap from 0.126s to 0.084s. The physics equation provides the structure; the Kalman filter provides the adaptation. Neither alone is sufficient.

[Interviewer is testing: Does the candidate understand the relationship between domain knowledge and data-driven adaptation?]

Q: Your model uses fixed coefficients like 0.114 s/lap for Soft tyres. Where did those numbers come from? Are they fitted to data?

A: These are calibrated from historical F1 telemetry analysis using multiple seasons of timing data. The base degradation rates (0.114 for Soft, 0.078 for Medium, 0.042 for Hard) represent clean-air, thermally-stable, reference-condition slopes -- they're the compound manufacturer's expected performance envelope. In a production system, these would be fitted per-weekend during Friday practice sessions (FP1/FP2/FP3) using the FastF1 practice data pipeline. For the competition prototype, they're literature-calibrated constants. The critical insight is that the EXACT value of baseDegMap doesn't matter as much as you'd think -- the Kalman filter adapts the effective rate online. Even if the base rate is wrong by 30%, the filter converges within 5-7 laps. The base rate just determines how fast convergence happens.

[Interviewer is testing: Does the candidate understand calibration vs fitting vs online adaptation?]

Q: Why didn't you use a proper database? Everything seems to be in TypeScript files.

A: Deliberate architectural choice for this competition prototype. The data shapes are defined once in types.ts and the data lives in TypeScript files (mockRaceData.ts, circuitTrackData.ts, realWorldValidationBahrain.ts) because: (1) It eliminates runtime database latency -- all data is compiled into the JavaScript bundle and accessed in O(1) from memory. (2) It ensures type safety at compile time -- if I change a field in LapRecord, the TypeScript compiler catches every consumer that breaks, which wouldn't happen with a schemaless database. (3) The total dataset is ~150KB -- it doesn't need a database. For production, I'd use Cloud Firestore (already integrated for strategy_runs) or a time-series database like InfluxDB for high-frequency telemetry. The important thing is the RaceDataProvider interface -- switching from in-memory to database requires implementing one interface, not refactoring the UI.

[Interviewer is testing: Architecture judgment -- can the candidate justify simple solutions for appropriate scales?]

Q: How would you handle this if the data were 100x larger -- say, all 20 drivers across all 24 races?

A: Three changes. First, the data layer moves from TypeScript constants to a database -- Firestore for race metadata and strategy audit trails, and a time-series database (InfluxDB or TimescaleDB) for sub-second telemetry streams. Second, the Kalman filter becomes driver-specific -- instead of a single scalar state, I'd maintain 20 parallel filter instances, one per driver, each with compound-specific sub-states. The state space grows from 2 to $2 \times 20 \times 3$ compounds = 120 scalars, which is still trivially cheap to compute. Third, the Monte Carlo engine moves to a Web Worker to avoid blocking the main thread -- 500 iterations x 20 drivers x 3 plans = 30,000 simulations per call, which could take 200ms+ on the main thread. Web Workers provide true parallelism in the browser. The architecture already supports this -- runStrategySimulation is a pure function with no DOM access.

[Interviewer is testing: Can the candidate reason about scale? Do they understand computational bottlenecks?]

Q: Your Monte Carlo uses Math.random() which isn't cryptographically secure. Does that matter?

A: No, and this is an important distinction. Cryptographic randomness (crypto.getRandomValues) is designed to be unpredictable to adversaries. Monte Carlo simulation needs statistical randomness -- uniform distribution, independence between samples, and a long period before the sequence repeats. Math.random() uses the xorshift128+ algorithm in V8, which has a period of 2^{128} and passes statistical test suites like TestU01. For 500 iterations, we generate ~1,500 random numbers -- we'd need to generate 10^{37} before the period repeats. Using crypto.getRandomValues would add overhead (system call to the OS entropy pool) with zero benefit for simulation correctness. It would only matter if an adversary could predict our random sequence to manipulate strategy recommendations -- which is not a threat model for a pit wall tool.

[Interviewer is testing: Distinction between statistical and cryptographic randomness. Practical engineering judgment.]

Q: What would you change if you had another month to work on this?

A: Three priorities. First, I'd run the FastF1 validation script on 10 more 2024 races to build a proper multi-race confidence interval and retune the P10-P90 sigma parameters to achieve target 80% coverage. Second, I'd implement a WebSocket connection to the FIA's live timing stream for real-time race monitoring -- the LiveOpenF1Provider skeleton already exists, it just needs the streaming protocol implementation. Third, I'd add compound-specific Kalman filter banks -- separate state estimates for Soft, Medium, and Hard -- so that data from one compound informs the next stint's predictions more intelligently. If I had more time beyond that, I'd implement safety car injection in the Monte Carlo engine using a Poisson process calibrated from historical safety car frequency data.

[Interviewer is testing: Self-awareness, prioritization, and realistic scope management.]

Q: Why should I hire you based on this project?

A: This project demonstrates four things that are hard to fake: First, domain-specific mathematical modeling -- I implemented a Kalman filter, Gaussian CDF, Monte Carlo engine, and heteroscedastic uncertainty model from scratch in TypeScript, not by importing a library. Second, scientific rigor -- the two-pillar validation with zero-leakage blind replay and LORO cross-validation is the same methodology used in peer-reviewed ML research. I know how to validate, not just build. Third, full-stack engineering -- one person built the React frontend, Express/FastAPI backends, TypeScript math engines, Firebase persistence, FastF1 integration, and 9 interactive modules with custom SVG visualizations. Fourth, intellectual honesty -- I can tell you exactly where the model fails (under-dispersed P10-P90, Silverstone/Spa synthetic data limitations, single-compound Kalman) and what I'd do to fix it. The codebase is the proof. Everything I just said is traceable to specific files, functions, and numbers.

[Interviewer is testing: Self-advocacy, confidence, and the ability to synthesize a project narrative.]

QUICK REFERENCE CARD

Keep these numbers memorized for rapid-fire interview questions:

Core Equation: 6-term latent lap-time decomposition

Soft Base Deg Rate: 0.114 s/lap

Medium Base Deg Rate: 0.078 s/lap

Hard Base Deg Rate: 0.042 s/lap

Fuel Compensation: -0.0581 s/lap (~1.72 kg/lap burn)

Track Evolution: -0.038 s/ln(lap)

Thermal Multiplier: +3.9%/deg C above 38 deg C

Kalman Initial State: $d_0=0.078$, $b_0=0.0$, $P_0=0.04$, $Q=0.0015$, $R=0.018$

Monte Carlo Iterations: 500 (default, range 50-2000)

Pit Loss (Monza): 21.4 seconds stationary

Cliff Onset (Soft): Mean 15.5 laps, sigma 1.2

Cliff Onset (Medium): Mean 24.0 laps, sigma 1.4

Cliff Onset (Hard): Mean 35.0 laps, sigma 1.8

Inference Latency: 1.4 ms per lap prediction

Internal MAE (Pillar A): 0.084 s (R-squared 0.914)

Bahrain MAE Sainz (Pillar B): 0.574 s (unseen circuit)

Bahrain MAE Verstappen: 0.420 s

Kalman Improvement: +49% to +65% vs offline model

Pit Stop Accuracy: 100% within +/- 1 lap of actual

P10-P90 Z-score: 1.282 (80% central coverage)

Base Residual Sigma: 0.012 s/lap

Simulation Tick Rate: 40ms / 25fps

Base Lap Duration (sim): 1600ms at 1x speed

Dashboard Modules: 9 (Command, Tyres, Strategy, Live, Stint, Track, Reports, Diagnostics, Validation)

Tech Stack: React 19 + TS 5.8 + Vite 6 + Tailwind 4 + Express 4 + FastAPI + FastF1 + Firebase + Gemini

Circuits Supported: Monza, Silverstone, Spa, Bahrain

Frontend Components: 5 shared + 9 page modules

Total Type Definitions: 14+ interfaces in types.ts

Firestore Project: truewear-29356

GitHub: github.com/hackChinmay/truewear-tyre-degradation